

PHAETON CAPITAL

TECHNICAL ASSESSMENT & ROADMAP

Prepared for: Antigravity Development Team

Date: 29 March 2026

Specialists: Quant Analyst · Backend Engineer · Frontend Engineer · Data/AI Pipeline

CLASSIFICATION: INTERNAL — DO NOT DISTRIBUTE

EXECUTIVE SUMMARY

Phaeton Capital has the right vision, an exceptional visual design, and a technically ambitious foundation. Four specialist reviews were conducted in parallel across quantitative finance, backend architecture, frontend/UX, and data/AI pipeline. The findings are consistent: the system creates the **appearance of predictive rigour** without the substance. Several models contain mathematical errors. The data pipeline silently loses records in production. The AI sentiment pipeline has no calibration. The frontend displays hardcoded fictional market data.

This document details every issue found, its severity, and the exact fix required. It is structured as a prioritised engineering backlog. Nothing here requires an architectural rewrite — these are targeted, concrete fixes that can be completed in two sprints.

Area	Grade	Core Problem
HMM Regime Detection	F	Lookahead bias — trained and tested on the same data
Hurst Exponent	D	Formula mathematically wrong; lag range 2–20 days is too short
Kelly Criterion	D	Formula is mean/var — this is not Kelly criterion
Monte Carlo	C-	Correct structure; naive GBM; no jumps, no vol clustering
GARCH / Volatility	B-	Mostly correct; needs Student-t dist and convergence checks
Sentiment Pipeline	D+	Overstated weight; LLM confidence uncalibrated; no test set
Technical Indicators	B	Calculations correct; misused (lagging signals treated as leading)
Fundamental Analysis	C	Basic metrics only; P/E not growth-adjusted; no quality screens
Macro Framework	C-	VIX/yield concepts sound; fear/greed index is ad-hoc
Recommendation Engine	D	Arbitrary weights; no backtest; no ablation; no alpha proof
Backend Architecture	D+	Race conditions; silent data loss; Python zombie processes
Data Pipeline	D	Bot-contaminated; RSS fragility; no authentication; no fallback
AI Pipeline	D+	No ground truth; no prompt versioning; fallback silently changes model
Frontend / UX	C+	Exceptional design; fake live data; 987-line god component; no mobile

1. QUANTITATIVE MODELS

The app implements HMM, Hurst exponent, Monte Carlo, Kelly Criterion, GARCH, Granger causality, Bayesian aggregation, Ornstein-Uhlenbeck, and ADF stationarity tests. Several contain fundamental errors. None have been validated out-of-sample.

1.1 Hidden Markov Model — Lookahead Bias

File: apps/worker/src/advanced_math.py lines 102–116

```
model.fit(returns) → model.predict(returns) # same data = data leakage
```

The model is trained and tested on identical data. It will always appear to identify regimes correctly while being useless on future data. 126 trading days is also too short for regime structure to stabilise.

- **Fix:** Expanding-window training — fit on data up to day T, predict day T+1 only.
- **Fix:** Use 3–4 states with economically labelled transition matrices.
- **Fix:** Remove random_state=42 from production code.
- **Fix:** Require minimum 2 years of daily data before running.

1.2 Hurst Exponent — Wrong Formula

File: apps/worker/src/advanced_math.py lines 118–128

```
lags = range(2, 20) # too short — minimum 50+ required  
hurst_exponent = poly[0] * 2.0 # constant is unjustified
```

The scaling formula applies $\sqrt{\text{std}}$ where raw variance is required. The lag range 2–20 is far below the minimum (50+) needed for the estimate to stabilise. The $\times 2.0$ multiplier is unexplained.

- **Fix:** Implement DFA (Detrended Fluctuation Analysis) with lags from 10 to $N/4$.
- **Fix:** Detrend the price series before computing.
- **Fix:** Require minimum 500 observations; flag result as inconclusive if CI spans 0.3–0.7.

1.3 Kelly Criterion — Mathematically Incorrect

File: apps/worker/src/advanced_math.py lines 130–137

```
kelly_fraction = mean_ret / var_ret # NOT Kelly # Correct: f* = (bp - q) / b where b=odds,  
p=win_prob, q=loss_prob
```

The formula computed is mean/variance. Kelly requires win/loss odds and probabilities. Full Kelly on 6 months of non-stationary returns also implies ~40% max drawdown even with edge.

- **Fix:** Implement correct continuous Kelly (μ/σ^2) with explicit documentation of assumptions.
- **Fix:** Apply fractional Kelly $\times 0.25$ to limit drawdown.
- **Fix:** Subtract estimated transaction costs from μ before computing.

1.4 Monte Carlo — No Fat Tails

File: apps/worker/src/advanced_math.py lines 165–208

```
shock = sigma * np.random.normal() # thin-tailed — underestimates downside
```

- **Fix:** Replace normal shocks with Student-t (df=4–6) to capture fat tails.
- **Fix:** Propagate GARCH vol forward through each path — not just today's vol for all 15 days.
- **Fix:** Add Poisson jump component for earnings/news gap risk.

1.5 Granger, TE, ADF — Computed but Discarded

Granger p-value, transfer entropy, ADF stationarity, and OU calibration are all computed in advanced_math.py and stored, but recommendation.ts never reads them. The ADF test on bounded [-1,1] sentiment is trivially stationary — it adds nothing.

- **Fix:** Incorporate Granger p-value into signal score (downweight sentiment when $p > 0.1$) or delete the computation.
- **Fix:** Delete ADF stationarity test on sentiment.
- **Fix:** Replace transfer entropy with Pearson correlation — simpler and more interpretable.

1.6 Recommendation Weights — Arbitrary Guesses

```
sentiment:0.25 technical:0.20 fundamental:0.20 quant:0.20 insider:0.10 macro:0.05
```

No backtest, no regression, no factor study proves sentiment is 5× more important than macro. These are guesses codified as science.

- **Fix:** Run 2-year walk-forward backtest. Use ridge regression to learn optimal weights from prediction outcomes.
- **Fix:** Add regime-conditional weights: in high-VIX, macro → 0.30, sentiment → 0.10.
- **Fix:** Ablation study: measure Sharpe with and without each signal source.

2. BACKEND ARCHITECTURE

The worker is a monolith coordinating scraping, AI batching, DB writes, Python subprocesses, risk computation, and scheduling in a single process with in-memory state. It will fail silently and frequently in production.

2.1 Race Conditions & Silent Data Loss

File: apps/worker/src/index.ts lines 196–210 and 285–313

```
// findUnique() then create() – another worker inserts same hash in between // P2002 unique constraint fires → record silently dropped, no retry
```

- **Fix:** Use createMany({ skipDuplicates: true }) — one atomic operation eliminates the race entirely.
- **Fix:** Wrap all financial history upserts in prisma.\$transaction() — currently a crash mid-loop leaves permanent partial data.
- **Fix:** Log every skipped record to an audit table with reason code.

2.2 Python Subprocess Resource Leak

Files: fundamentals.ts, quant.ts, macro.ts

```
Bun.spawn(["python3", ...]) // spawned per request, no guaranteed cleanup setTimeout(() => proc.kill(), 30000) // SIGTERM often ignored by Python
```

Three modules each spawn a Python process per request. After thousands of calls, zombie processes accumulate until the system exhausts file descriptors and crashes.

- **Fix:** Replace all three Python callers with a single persistent FastAPI service exposing /fundamentals, /quant, /macro.
- **Fix:** Add to docker-compose.yml as a dedicated python_worker service. Call over HTTP from TypeScript.
- **Fix:** This also eliminates stdout buffering issues, JSON split reads, and timeout ambiguity.

2.3 In-Memory Job Queue — Lost on Restart

File: apps/worker/src/index.ts lines 17–19

```
let jobQueue: string[] = []; // lost on pod restart let isRunning = false; // not thread-safe
```

- **Fix:** Replace with BullMQ backed by the Redis instance already in docker-compose.yml.
- **Fix:** Add dead-letter queue with exponential backoff for failed jobs.
- **Fix:** De-duplicate jobs: same ticker within 10 minutes should not re-queue.

2.4 500 Individual DB Inserts Per Scrape

- **Fix:** Collect all validated sentiment objects, call prisma.sentiment.createMany() once per batch.
- **Fix:** Set explicit pool in DATABASE_URL: ?connection_limit=20&pool_timeout=30

2.5 Zero Observability

- **Fix:** Add /health and /ready HTTP endpoints for container orchestration.
- **Fix:** Replace console.log/error with structured JSON logging (pino) with correlation IDs.
- **Fix:** Expose Prometheus metrics: sentiment_saved_total, ai_batch_duration_ms, queue_depth, python_errors_total.

3. DATA SOURCES & AI PIPELINE

3.1 Reddit — Trust Filter Removed

File: apps/worker/src/scrapper.ts lines 54–55

```
author_karma: 500, // HARDCODED FAKE — comment says 'removed per audit' author_age_days: 100, // HARDCODED FAKE
```

Every Reddit post — bots, pump-and-dump campaigns, spam — is now ingested as valid signal. The entire sentiment pipeline is contaminated at the source.

- **Fix:** Implement Reddit OAuth2 and use the authenticated API (real karma, real age).
- **Fix:** Soft-score trust: karma 100–500 maps linearly to 0.3–1.0 weight multiplier.
- **Fix:** Detect coordinated bursts: >20 similar posts within 1 hour → flag as manipulation.

3.2 3 of 5 Sources Are Unofficial RSS

Google News RSS and Yahoo Finance RSS are undocumented endpoints. Google has deprecated RSS feeds before without notice. No fallback exists if a source goes down.

- **Fix:** Replace Google News RSS with NewsAPI.org (free tier: 100 req/day) or Polygon.io News.
- **Fix:** Add source-health monitoring: 0 results for 3 consecutive cycles → alert.
- **Fix:** Tag every sentiment record with its source for credibility-weighted aggregation.

3.3 Gemini Sentiment — No Calibration

LLM confidence scores are used directly. There is no test set, no F1 score, no calibration curve. An LLM claiming 0.95 confidence does not mean 95% accuracy. You don't know if this is better than random.

- **Fix:** Label 500 posts manually. Measure Gemini precision/recall against labels.
- **Fix:** Apply Platt scaling or isotonic regression to calibrate confidence scores.
- **Fix:** Consider FinBERT as supplement — domain-specific model with published accuracy benchmarks.
- **Fix:** Add prompt_version column to Sentiment table. Bump on every prompt change.

3.4 Fallback Model Changes Signal Silently

File: apps/worker/src/ai.ts lines 94–128

```
On 429 → instantly retry with gemini-1.5-flash → ai_model field updated // But Bayesian aggregation treats both models as equivalent. They are not.
```

- **Fix:** Add exponential backoff with jitter before trying fallback.
- **Fix:** Apply a calibrated discount factor (e.g. 0.7×) to fallback model scores until equivalence is empirically proven.
- **Fix:** Track fallback rate as a metric — >20% of batches falling back = primary quota insufficient.

3.5 EDGAR — Metadata Only, No Content

- **Fix:** Use SEC EDGAR Full-Text Search API to retrieve filing content.
- **Fix:** Apply LLM sentiment to filing text, weighted at 3× Reddit (higher credibility tier).
- **Fix:** Flag 8-K filings containing material keywords: restatement, CEO, SEC, investigation, default.

3.6 Critical Missing Data Sources

- **Options Flow:** Put/call ratio and unusual options volume — leading institutional indicator. Source: CBOE free data or Unusual Whales API.
- **Short Interest:** Borrow rate spikes predict squeezes. Source: FINRA short interest reports (free, bi-weekly).
- **Earnings Transcripts:** Q&A; tone is highly predictive. Source: SEC EDGAR 8-K full text or Motley Fool API.
- **Analyst Revision Momentum:** Are targets rising or falling? Capture upgrades_downgrades from yfinance.
- **Dark Pool / Block Trades:** Institutional positioning. Source: FINRA OTC Transparency data.

4. FRONTEND & UX

4.1 Hardcoded Fake Market Data — Remove Immediately

File: apps/web/app/DashboardClient.tsx lines 327–333

```
NVDA +2.4% TSLA -1.2% // fictional, hardcoded, displayed as live
```

This is fictional data displayed as live market movement on a financial application. This is the single most damaging credibility issue in the entire codebase.

- **Fix:** Delete immediately. Either fetch real price data (Polygon.io, yfinance JS) or remove the ticker tape.

4.2 Bootstrap CI Bands — Computed but Never Shown

File: apps/web/app/page.tsx lines 79–80

1,000 bootstrap iterations run on every page request. lowerBound + upperBound computed, passed as props, and never rendered. The most statistically honest feature in the app is invisible while burning CPU on every load.

- **Fix:** Render lowerBound and upperBound as a shaded ReferenceArea on the Gaussian distribution chart.
- **Fix:** Move bootstrap computation to the worker — pre-compute and store in DB, not on every page render.

4.3 987-Line God Component — all typed 'any'

File: apps/web/app/DashboardClient.tsx

```
export default function DashboardClient({ ...14 props... }: any) // zero type safety
```

- **Fix:** Split into: SentimentPanel, FundamentalsPanel, TechnicalPanel, QuantPanel, InsiderPanel, MacroPanel, RiskPanel, RecommendationPanel.
- **Fix:** Define TypeScript interfaces for all props. Remove every instance of 'any'.

4.4 force-dynamic + 7 DB Queries on Every Request

- **Fix:** Change revalidate=0 to revalidate=60 — data only updates every 15 minutes.
- **Fix:** Cache fundamental data in Redis with 5-minute TTL.
- **Fix:** Replace 30-second client polling with Server-Sent Events — push updates only when new sentiment arrives.

4.5 Language Inconsistency

Section labels are in Swedish (Helhetsanalys, Insynshandel) while navigation and errors are English. Pick one language. English is recommended for broader market reach.

4.6 Mobile Is Broken

- Search bar is hidden md:block — invisible on phones.
- Sidebar hover-expansion does not work on touch.

- **Fix:** Add bottom navigation bar for mobile. Test on iPhone SE (375×667).

4.7 Missing Key Visualizations

- **Price chart with sentiment overlay** — the core product vision; completely absent.
- **Prediction audit trail** — last 20 predictions vs. actual outcomes with hit rate and Sharpe.
- **Volatility regime indicator** — animated state machine (Calm / Normal / Stressed / Crisis).
- **Insider trade price-impact chart** — annotate price chart with insider events; show what happened next.
- **Sector/peer percentile ranking** — 'This stock is 85th percentile in its sector by momentum.'

5. WHAT WILL TAKE THIS TO THE NEXT LEVEL

Beyond fixing the issues above, these five features separate a prototype from a tool people actually rely on.

① Price + Sentiment Causality Chart (Highest Impact)

Show stock price over time with the rolling sentiment score overlaid on the same axis. Annotate moments where Granger causality was statistically significant. This makes the entire ML pipeline visible and verifiable. It is also the most compelling demo possible — users can see whether sentiment led price in the past.

② Regime-Aware Signal Weighting

Change which signals are highlighted based on current market regime. Trending market → emphasise momentum and MACD. Ranging market → RSI mean-reversion and Bollinger bands. Crisis ($VIX > 30$) → suppress technical signals, amplify macro and liquidity. This is what separates a system from a dashboard.

③ Prediction Audit Trail

Show users the last 20 predictions for the current ticker and what actually happened: hit rate, average return, maximum loss, Sharpe ratio. If the model is right 55% of the time, show it. If 40%, show that too. This is the single most trust-building feature possible.

④ The 'Why' Narrative — LLM Synthesis

Replace the wall of 50 metrics with 3 sentences generated by the LLM: 'RSI is oversold at 28. Insiders purchased \$2.1M in the last 14 days. Sentiment turned positive after the earnings beat. Model confidence: 68%.' This is what separates a decision tool from a data museum.

⑤ Options Flow Integration

Put/call ratio and unusual options activity is the single most powerful leading indicator available publicly. Unusual call buying at strikes 10–20% above current price signals institutional bullish positioning before it appears in price. Source: CBOE free feed or Unusual Whales API. Add this and you have something institutional traders actually pay for.

6. PRIORITISED ACTION TABLE

P0 — Before Anything Else

Priority	Fix	Why
■ P0	Delete hardcoded fake ticker tape	DashboardClient.tsx:327–333 — fictional data on a finance app destroys credibility
■ P0	Restore Reddit OAuth authentication	scraper.ts:54–55 — trust filter gutted; entire signal is bot-contaminated
■ P0	Wrap all related DB writes in prisma.\$transaction()	index.ts:128–136 — crash leaves permanently incomplete financial history
■ P0	Replace in-memory jobQueue with BullMQ on Redis	index.ts:17–19 — every pod restart loses all queued work
■ P0	Replace individual create() loop with createMany(skipDuplicates)	index.ts:285–313 — 500 round-trips + race condition drops records silently

P1 — First Sprint

Priority	Fix	Why
■ P1	Fix HMM: expanding-window (fit T, predict T+1)	advanced_math.py:102–116 — lookahead bias makes it actively misleading
■ P1	Fix Kelly formula (mean/var is not Kelly)	advanced_math.py:130–137 — wrong math in a financial app
■ P1	Add prompt_version column to Sentiment table	ai.ts — every prompt change makes historical scores incomparable
■ P1	Render bootstrap CI as shaded bands on distribution chart	page.tsx:79–80 — computed, burns CPU, never shown
■ P1	Convert Python subprocesses to persistent FastAPI service	fundamentals.ts, quant.ts, macro.ts — zombie process accumulation
■ P1	Add structured JSON logging + correlation IDs	All worker files — production outages are currently invisible
■ P1	Add /health and /ready endpoints to worker	Required for container orchestration and incident response

P2 — Second Sprint

Priority	Fix	Why
■ P2	Label 500 posts; calibrate Gemini confidence scores	ai.ts — unknown if LLM scores are better than random

■ P2	Build price + sentiment overlay chart	DashboardClient.tsx — the killer feature; completely absent
■ P2	Replace Google/Yahoo RSS with NewsAPI or Polygon.io	scraper.ts — unofficial RSS can disappear overnight
■ P2	Parse EDGAR filing content, not just metadata	scraper.ts:154–189 — material events look identical to routine filings
■ P2	Fix Hurst exponent: DFA with 50+ lags on detrended series	advanced_math.py:118–128 — current formula is wrong
■ P2	Monte Carlo: Student-t shocks + GARCH path propagation	advanced_math.py:165–208 — GBM underestimates downside
■ P2	Commit to English throughout the frontend	Mixed Swedish/English is unprofessional
■ P2	Fix mobile navigation	App is unusable on phones

P3 — Strategic

Priority	Fix	Why
■ P3	Add options flow data (CBOE or Unusual Whales)	Biggest single signal quality upgrade available
■ P3	Build walk-forward backtest pipeline (5 years, annual retest)	No evidence any model outperforms buy-and-hold without this
■ P3	Build prediction audit trail UI	Most trust-building feature; almost no retail tool does this honestly
■ P3	Optimise recommendation weights via ridge regression on outcomes	recommendation.ts:3–10 — current weights are guesses
■ P3	Regime-aware signal weighting (high-VIX → amplify macro)	Makes the system adaptive to market conditions
■ P3	Replace polling with Server-Sent Events	Push updates only when new data arrives; eliminate wasted polling
■ P3	Add LLM 'Why' narrative synthesis per ticker	Replace 50 metrics with 3 sentences answering: should I buy?

7. PRODUCTION FAILURE FORECAST

If P0 and P1 items are not addressed, the following failures are predictable in order of occurrence:

When	Failure	What Happens
Week 1	DB connection pool exhaustion	500 individual inserts per scrape cycle exhaust the default pool. Worker appears alive but writes nothing.
Week 1–2	Python zombie process accumulation	SIGTERM does not reliably kill Python. After ~10,000 spawns the system runs out of process slots. Worker crashes.
Week 2	Sentiment loss from race conditions	Parallel workers cause duplicate hash collisions. Records silently dropped. AI spend wasted. Backtest sample biased.
Week 2–3	AI rate limit cascade	Token estimation off by 2–5× for emoji-heavy posts. Primary 429s. Fallback also 429s. Entire batch discarded.
Month 1	Prediction accuracy data corruption	Prediction updates without transactions. Partial writes on crash corrupt accuracy metrics. False 95% accuracy reported.
Month 1+	Bot-contaminated signal degrades recommendations	Without auth + trust filters, coordinated P&D; campaigns flood the DB. App follows manipulation, not fundamentals.

8. CLOSING ASSESSMENT

Phaeton Capital has the right vision, an exceptional visual design, and an architecturally sound foundation. The gap is between the surface appearance and the underlying rigour.

The quant models need correct implementation or removal. The data pipeline needs authentication, calibration, and reliability. The backend needs transactions, proper queuing, and observability. The frontend needs to remove fictional data and add the features that make the ML pipeline visible and trustworthy.

None of these are rewrites. They are targeted, concrete fixes — the P0 and P1 items can be completed in a single sprint. Once done, the app crosses from impressive prototype to credible product.

The price + sentiment overlay chart, the prediction audit trail, and the regime-aware signal weighting are the three features that will define whether this becomes a tool people rely on or a dashboard they visit once.

END OF ASSESSMENT

Phaeton Capital Technical Review — 29 March 2026